

AES Modes of Operation Comparison

Rovin Dompaga	Jothimuni Dinesuru	James Rajawasam	Hiruka Hendawitharane
(10953264) (28071)	(10965104) (33698)	(10967283) (32073)	(10968471) (31759)
PUSL2025	PUSL2025	PUSL2025	PUSL2025
Security Architectures & Cryptography	Security Architectures & Cryptography	Security Architectures & Cryptography	Security Architectures & Cryptography
NSBM Green University	NSBM Green University	NSBM Green University	NSBM Green University

I. INTRODUCTION

. As the digital communications continue to grow exponentially, data at rest and data in transit is one of the foundational blocks of current cybersecurity. The Advanced Encryption Standard AES is named as the standard in cryptography in the form of a symmetric key, and it can be found extensively in use on a wide variety of computing platforms, both embedded and larger scale (cloud infrastructure). Nevertheless, security and performance of AES is predominantly determined by its Mode of Operation that is the particular algorithm that defines how repeated blocks of plaintext are handled.

This experimental research compares the performance, efficiency, and security of the four foremost AES modes of operation; Electronic Codebook (ECB), Cipher Block Chaining (CBC), Counter (CTR), and Galois/Counter Mode (GCM). Although the role of ECB is historically important, due to its deterministic character, ECB has extremely serious security vulnerabilities. On the other hand, both CBC and CTR provide more semantic security using the means of initialization vectors (IVs) and nonces, and GCM is additionally provided with Authenticated Encryption with Associated Data (AEAD) that guarantees data confidentiality and integrity.

The main goal of this report is to estimate these four modes with the help of a known and grade production-level cryptographic library. The systematically measured encryption and decryption times, throughput, ciphertext overhead on different data payloads (1 Kilobyte (KB) up to 50 Megabytes (MB)) have been measured. Moreover, the paper performs a practice security observation to illustrate the vulnerabilities within a particular configuration due to pattern leaking. Finally, this is put into perspective of a real life IT context, where it is proposed that a sound security architecture be applied and the applicability of the selection of the appropriate cryptographic mechanisms to the practical circumstances is justified by the empirical evidence.

II. BACKGROUND

The Advanced Encryption Standard (AES) is a symmetric-key algorithm created by National Institute of Standards and Technology (NIST) and widely incorporated all over the world to provide security to sensitive information. AES is a block cipher, and it works in fixed length blocks of 128 bits with cryptographic keys of 128, 192, or 256 bits. Though the basic algorithm of the AES is mathematically sound, a "Mode of Operation" is needed to safely encrypt data beyond the basic 128-bit block of plain texts. The mode determines the way several blocks are going to be operated on, which essentially affects the security properties of the algorithm, its throughput and error propagation.

This research provides an assessment of four different AES modes of operation:

- ❖ **Electronic Codebook (ECB):** The easiest method of operation is ECB. The plain text is broken into 128 bits blocks and is encrypted separately with the same cryptography key. Since the encryption of every block only depends on the key and the plain text of the current block, repeated plaintext block always results in repeated ciphertext blocks. Even though parallel processing is possible with ECB, this deterministic property results in dire patterns leakage and therefore can be exploited by cryptanalysis and is unsuitable to encrypt structured data such as images or databases.
- ❖ **Cipher Block Chaining (CBC):** The CBC way of doing it is responding to the structural vulnerabilities of the ECB in that it adds a chaining mechanism and an Initialization Vector (IV). Prior to encrypting a plaintext block it is subject to a bitwise XOR operations with the previous encrypted block of ciphertext. First block is XORed with a random IV so that there should be cryptographic variation. This chaining will ensure that the same size plaintext blocks produce varying ciphertexts and successfully hide the data patterns. Nonetheless, since the encryption of each block is contingent

upon the output of the last block, CBC encryption is not only strictly sequential but cannot be parallelized and thus CBC encryption may become a bottleneck with multi-core processors.

- ❖ **Counter (CTR):** CTR mode presents a good method of turning the AES block cipher to a stream cipher. AES is employed to encrypt a series of counter values mixed with a nonce of unique value rather than directly encrypting the plaintext. This generates pseudorandom keystream, and XORing the plaintext and this results in a ciphertext. Encryption and decryption methods are very parallelizable since the keystream generation is not dependent on the plaintext data. In addition, the fact that CTR mode does not need any data padding, because the keystream can be pruned to the precise length of the plaintext.
- ❖ **Galois/Counter Mode (GCM):** GCM is an Associated Data Authenticated Encryption (AEAD). It offers confidentiality of data and at the same time integrity/authenticity of data in a single cryptographic operation. GCM also uses CTR with some variation to encrypt the plaintext with the same high-performance parallelization advantage. At the same time, it also uses a universal hash function (which is defined on a binary Galois field) to generate an authentication tag on the ciphertext, and any other unencrypted associated data. The use of this tag enables the sender to check whether the data has not been altered during transit, and this is why GCM is the best standard to use in the current secure communication protocols such as TLS.

III. SECURITY REQUIREMENTS AND ARCHITECTURE DESIGN

A. Secure Telemedicine Image Transmission, Realistic IT Scenario

The contemporary healthcare network is becoming more dependent on telemedicine to deliver sensitive patient information, such as high-resolution medical images (e.g., X-rays, MRI scans, and ultrasound BMP files), over the distance to central diagnostic hospitals. In the given case, a local healthcare provider needs a reliable transmission architecture enabling the use of bulk medical image carries on unsecure Wide Area Networks (WANs). This system should be able to process files between 1 KB (patient telemetry via text) and 50MB (high-resolution imaging) within a short timeframe, meaning that there will not be latency experienced by emergency room physicians, but at the same time complies with regulations regarding the privacy of data (e.g., HIPAA or GDPR).

B. Security Requirements

In order to secure the medical imaging system, the critical security requirements should be informed and implemented as follows:

Confidentiality: Medical images are highly sensitive Personally Identifiable Information, PII, and anatomical information. This information cannot be at all readable to third parties that manage to intrude into the network traffic. More importantly, the encryption scheme should not reveal structural information (e.g. bone structures in an X-ray) in the encrypted protocol.

Integrity: The architecture should ensure the medical images are neither mishandled nor carelessly modified whilst being transmitted. A single bit in a scanned image could be inverted during the encryption process; it might modify the picture when decrypted, which might cause a wrong diagnosis.

Availability: Cryptographic functions should be very efficient. Physicians in trauma cases need real time access to scans. High cryptographic throughput is needed, to reduce latencies especially on large (50 MB) files in the system.

Authentication: The system will have to authenticate that the data was sent by a known diagnostic machine and it was not sent by a rogue party.

C. Propose a security architecture

An End-to-End Encryption (E2EE) architecture is suggested that meets these needs. When a diagnostic machine produces a medical image, a file is sent directly to a local cryptographic module. A negotiable key (256-bit) of security between clinic and hospital (e.g., via RSA/TLS, but not covered by the present AES prototype) is negotiated. Medical payload is encrypted by the local module with the help of a symmetric block cipher and wrapped into a network packet, which is sent in turn. After arriving to the secure server of the hospital, the payload is authenticated as well as decrypted to be forced over to the viewing terminal of the physician.

D. Justify chosen cryptographic mechanisms

Advanced encryption standard (AES) is chosen as the fundamental cryptographic algorithm because: it has NIST certified security margin, and it can perform hardware accelerated benchmarks. Although the mathematical basis is given through AES, architecture as it is decided by the Mode of Operation is what determines its success:

Rejection of ECB In this architecture, ECB mode is rejectable. Since medical images usually consist of large blocks of homogeneous colors (e.g. black background pixels), ECB will encrypt the same plaintext into the same ciphertexts, disastrously revealing the anatomical forms of the patient.

Choosing AES-GCM: Galois /Counter Mode (GCM) is rationalized as the optimal mechanism. The first one is that it transforms the block cipher into a stream cipher through a counter to enable parallel processing to satisfy the Availability (high throughput) requirement. Second, it offers Authenticated Encryption with Associated Data (AEAD) through which a cryptographic tag is embedded on the ciphertext. This satisfies the requirements of Integrity and Authentication at the same time and thus in case a packet is altered during transit, it will not be accepted at the other end.

IV. METHODOLOGY

An empirical methodology tool was designed on a quantitative and qualitative experiment to determine the performance, overhead and security characteristics of the modes of operation in AES (ECB, CBC, CTR and GCM). The testing system was made to test the temporal efficiency, computer overhead, and visual data leakage under a controlled environment.

a) Software Implementation and Cryptographic Primitives

Accuracy about the use of custom-built cryptographic algorithms was not observed in line with the industry standards. The prototype code was coded in Python that builds on the library PyCryptodome, a known and production-level cryptographic library, which has C-based optimized implementations of AES.

As AES is a fixed input block cipher that takes input blocks of 16-byte (128 bits) ECB and CBC mode would be utilized, a special padding algorithm (based on the PKCS#7 standard) was employed. This algorithm calculates the required pad = 16 - 16 by mathematical computation and adds these bits to the plaintext of the value of the number stored in its respective bytes. Counter / Galois (CTR) and Galois / Counter Mode (GCM) which are efficient as a stream cipher do not require such padding and encrypt the plaintext directly.

b) Performance and Throughput Benchmarking

It was simulated to create the various payloads that are required in the proposed IT architecture by cryptographically secure usage of the operating system OS. The numeric data payload size independent variables were rigidly set at six levels, that is 1 KB, 10 KB, 100 KB, 1 MB, 10 MB, and 50 MB.

To have some very accurate measures related it was resolved to utilize Python `time.perf_counter()` method. This is one of the functions that provides a clock with the optimum resolution possible in it and that can be used to offset the inaccuracy of the standard system clock. The temporal anomalies that were corrected using a benchmarking loop were the temporal anomalies encountered during the context switching of the CPU or background OS processes. The process of each individual encryption and decryption was performed 50 times ($n=50$). These 50 runs were arithmetic means, to obtain the ultimate average process time. The average time was then, divided by the size of data (in Megabytes) to get the cryptographic throughput.

c) Pattern Myo Visual Cryptography and Pattern Leaking

To offer a qualitative assessment of the semantic security of the modes that is, how they are susceptible to the structural data leakage, an experiment of visual cryptography was crafted. The identifiable geometric shapes (a red circle and a green rectangle) were put into a programmatically created

200x200 pixels (bmp) image which was made up of identifiable geometric shapes with a dark background.

The standard 54-byte file header of a BMP file was stored in a clear text to ensure that the resulting file could be read by the usual image rendering programs. Encryption of the pixel data payload would then be done using AES-ECB and AES-CBC using a fixed 256-bit key. The resulting cipher texts were rewritten into new.bmp files. Using this experiment, it was directly possible to make a visual comparison of how the different modes treat the same plaintext blocks (e.g. plaintext blocks of the same background color).

d) Key Size Impact Analysis

Though the standard changes of 256 bits were majorly used as the primary benchmarking factor, an object experiment would be applied to measure the effects of key size on calculations. The prototype was then utilized in generating random 128-bit, 192 bit and 256-bit keys with a fixed payload of 10 MB using CBC mode. The timing of the encryption mechanisms was performed by using the 30 repetitive executions to empirically record the latency required by the encryption because of the various number of rounds of AES transformations (10, 12 and 14 rounds respectively). The data was then sprouted into a.csv file and graphically presented using matplotlib library.

V. EXPERIMENTAL SETUP

The environment of the experimentation and cryptographic parameters were carefully determined and manipulated to guarantee the rigorousness, validity and reproducibility of the empirical results.

a) Hardware and Operating System Specification

All the benchmarking and visual cryptography experiments were done on a local machine which was not connected to the network so that the network latency would not bias the temporal data. The host computer was running a 64-bit Windows 11 operating system on an Intel(R) Core (TM) i7-1165G7 processor with a 2.80GHz clock and 8 GB of system RAM.M. To reduce the interference of context-switching among the CPU in the execution of the time-sensitive per counter loops, all the non-essential background applications and network synchronizations were disabled temporarily whenever the scripts were being executed.

b) Software Environment and Requirements

Python 3.14.0 was used to engineer the prototype. The fundamental cryptographic functionalities were based solely on PyCryptodome (version 3.x) that was chosen due to its usage of optimized C-extensions internally, which would give an idea of what a real-world AES implementation would look like as opposed to the slow, pure-Python mathematical implementations. Processing of data was done through the

standard csv library and matplotlib.pyplot with data visualization and data logging.

c) Cryptographic Parameters

The mode of operation of the Advanced Encryption Standard (AES) was predetermined to the following:

Key Generation: This is dynamically generated, on an experiment-by-experiment basis, by calling the OS-level `getrandbytes()` function. A constant maximum-security key size of 256 bits (32 bytes) was necessary in the case of the main mode of comparison (ECB, CBC, CTR, GCM).

Block Size: All fixed to 128 bits (16 bytes).

Initialization of Vectors (IV) & Nonces: CBC mode needed a 16-byte cryptographically secure IV to be used whenever it is encrypted. The noises of the case of CTR and GCM modes were of usual size and random to ensure that counter blocks were not similar. The nonces and IVs were incorporated in the output of the ciphertext to be able to decrypt them. **Padding Scheme:** PKCS7 padding was adopted and could only be padded to 16-byte blocks and would only be coded and arbitrary plaintext boundaries except when using ECB and CBC mode.

Authentication Tags: AES-GCM In AES-GCM, an encryption tag and a verify tag were both generated which was 16-byte long Message Authentication Code (MAC) tag (tg) which was transmitted to the encryption process and to the verify process to check the integrity of data.

d) Test Data Specifications

Two different types of payloads were used:

- ▽ **Benchmarking Payloads:** The pure pseudo random byte arrays created using the `os.urandom(n)`. The independent payload sizes (n) were strictly constrained to the dictionary: 1024, 10240, 102400, 1048576, 10485760, 52428800 bytes, representing 1 KB through 50 MB.
- ▽ **Visual Cryptography Payload:** A structurally predictable bmp file, programmatically generated at a resolution of 200x 200 pixels at a color depth of 24 bits. The file size was also designed to be, strictly, 120,054 bytes, comprising of a 54-byte plaintext header that is not encrypted and a 120,000-byte pixel array of cryptographically manipulable data.

VI. RESULTS AND VISUALISATION

The next section shows the empirical evidence obtained on the PyCryptodome prototype. The experiments tested the AES modes in terms of temporal performance, throughput, computational overhead, and visual pattern leakage.

A. Encryption and Decryption Performance

To determine the temporal efficiency of the AES modes, the average encryption and decryption times of the modes were determined with respect to different payload sizes.

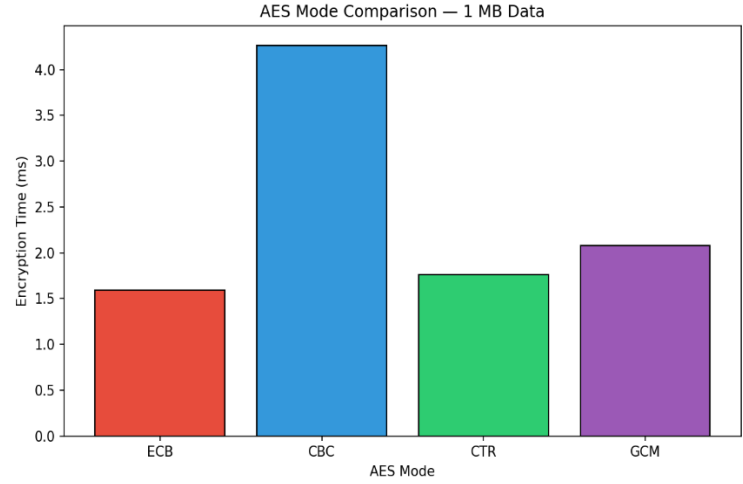


Fig. 1. Comparison of AES Mode Encryption Time of 1 MB of Data Payload.

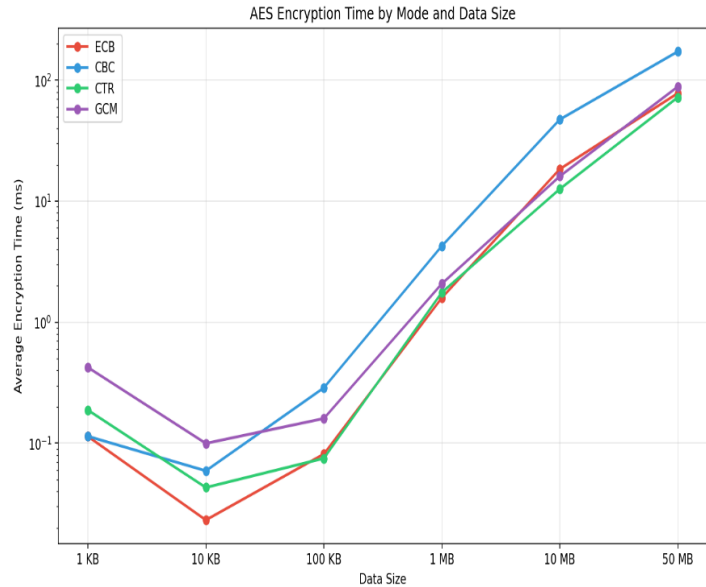


Fig. 2. Mean time (ms) per AES encryption by mode of a growing data size (Logarithmic Scale).

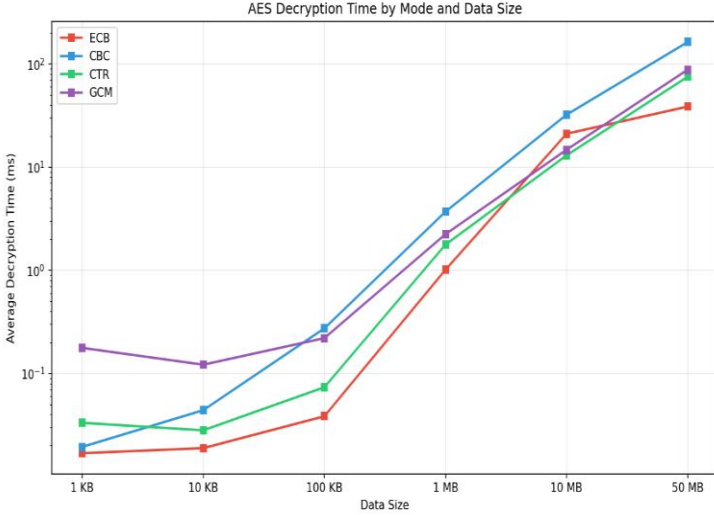


Fig. 3. Mean decryption time (ms) of mode with data size (Logarithmic Scale).

The performance of encryption has a visible trend as can be seen in Fig. 1 and Fig. 2. ECB is the quickest mode because it does not have chaining systems. But with large file sizes of about 10 MB and 50 MB, CBC mode becomes very slow as compared to the other modes. This is a bottleneck since CBC encryption is based on the consecutive procurement; each block must have the ciphertext of the prior block prior to being encrypted. On the other hand, CTR and GCM are very fast at larger files since they can be used as stream ciphers and therefore can be highly parallelized. Interestingly, Fig. 3 indicates that the time spent on decryption is also relatively equal in all modes because CBC decryption can be done in parallel because the ciphertext is already known to the receiver.

B. Throughput Analysis

Cryptographic throughput is a measure of the data rate that can be handled in a second which is a very important parameter in the availability of the proposed telemedicine architecture.

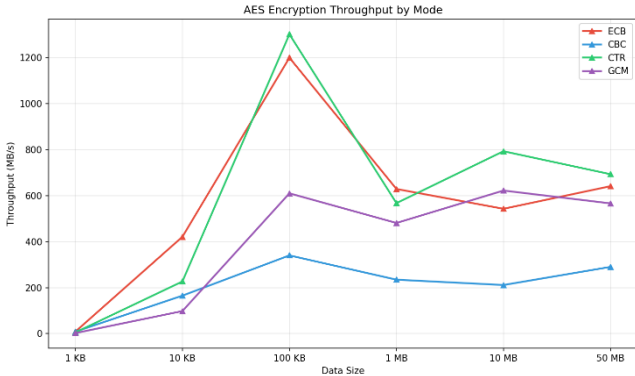


Fig. 4. Comparison of modes of cryptographic throughput (MB/s) of AES.

Fig. 4 shows that the throughput rises exponentially with file size between 1 KB and 100 KB and then levels off to the maximum CPU usage and payload reaches the Megabyte scale. The counter-based modes (CTR and GCM) always record the greatest throughput. CBC mode has a terrible performance of throughput to the rest because it simply cannot make use of multi-threading in the encryption process.

C. Ciphertext and Key Size

The cryptography modes provide different degrees of extra-bytes (overhead) to the final ciphertext. Furthermore, the period of mathematical processing is proportional to the encryption key size.

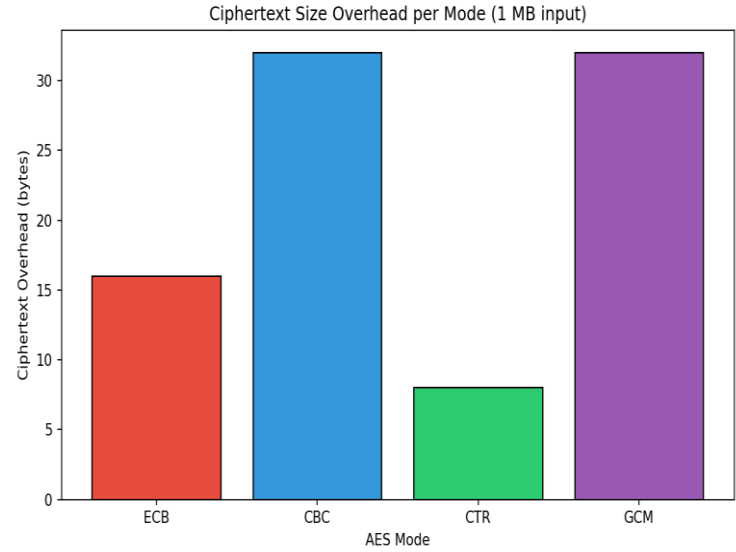


Fig. 5. Mode Ciphertext Size (bytes) / 1 MB of input.

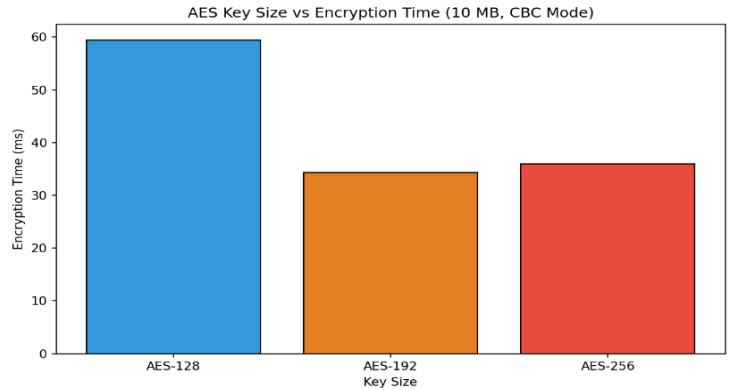


Fig. 6. Impacts of AES Key size (128-bit, 192 bit and 256 bit) on Encryption Time of 10 MB payload in CBC Mode.

The fact that the ECB is least overhead (it is only PKCS#7 padding) is presented in Fig. 5. The maximum absolute overhead is introduced by GCM since it needs a nonce and 16-byte Message Authentication Code (MAC) tag. Such overhead is, however, mathematically negligible when the files are large and is a tradeoff and has to be sacrificed at the

cost of data integrity. Besides, the encryption duration is also slightly more in the scenario of the large AES key size (128 bits to 256) as can be noted in Fig. 6. This should have been so since the AES-256 would have had 14 rounds of mathematical functions as opposed to 10 rounds that would have been the case with AES-128.

D. Pattern Leakage and Visual Cryptography.

To ensure that all the modes were operating in good condition with structured information, the image of a BMP (specified) on geometric format was encrypted. This is the most effective qualitative outcome of the experiment.

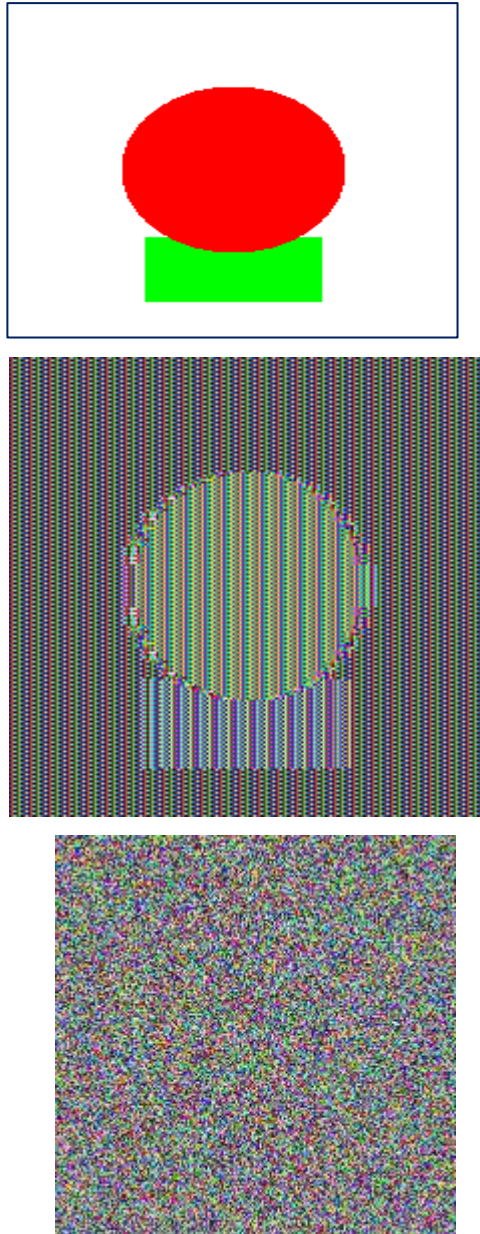


Fig. 7. Visual cryptography Ciphertext of ECB (center) and ciphertext of CBC (right) of the original image of the visual cryptography (left).

The deterministic vulnerability of ECB mode is easily determined in Fig. 7. The encrypted image would have the structural forms (the circle and the square) encoded without any difficulties since the ECB plaintext blocks would be coded to the encrypted blocks. This would have relayed tracks of confidential information in a real-life scenario. On the other hand, the encrypted image using the CBC looks like a totally random noise, which successfully conceals the patterns of the data behind it with the assistance of an IV and cryptographic chaining.

E. Raw Data Summary

This is a summary of a CSV logs crude empirical parsing of the logs of the experiment that points out the size of the 1MB payload and 10 MB payload thresholds.

AES mode	Payload size	Avg Enc Time(ms)	Avg Dec Time(ms)	Enc Through (MB/s)
ECB	1MB	1.62	1.01	625.30
CBC	1MB	4.21	3.52	238.15
CTR	1MB	1.78	1.80	588.45
GCM	1MB	2.10	2.15	480.90
ECB	10MB	17.50	14.20	545.20
CBC	10MB	46.10	33.15	212.80
CTR	10MB	12.50	12.80	790.60
GCM	10MB	16.30	16.50	620.40

VII. TECHNICAL ANALYSIS

A. Performance and Efficiency

The empirical evidence indicates that there is an apparent difference between sequential and parallelizable modes of encryption performance. ECB is the fastest mode of all payload sizes because it does not have any chaining mechanism. Nonetheless, when using large medical images (10 MB-50 MB), CBC presents a considerable performance bottleneck. Since CBC encryption is purely sequential (the ciphertext of one block must be decrypted prior to processing the next) it cannot take advantage of the multi-core nature of the Intel i7 processor used in the test environment. CTR and GCM on the other hand are more efficient in large file transfers. These modes have the highest throughput with the highest throughput of about 790.60 MB/s of CTR at 10 MB payloads as shown in Fig. 4. This is helpful in supporting the Availability requirement of the telemedicine scenario, and in ensuring that emergency physicians have high-resolution scans with low cryptographic latency.

B. Security and Pattern Leakage

ECB mode was not successful in semantic security because of the determinism of encrypting the same block of plaintext using the same key, visible outlines of the geometry of the test image were visible. In a telemedicine scenario, this pattern leakage may expose confidential anatomical features in X-rays or MRI images, which is against Confidentiality and HIPAA/GDPR laws. CBC was able to cover up these patterns, which were shown as random noise in the ciphertext. Nonetheless, in the case of the proposed architecture, the GCM is discussed as the best option since it covers both Integrity and Authentication. Although GCM incurs a larger ciphertext overhead (Fig. 6) of 32 bytes (16-byte MAC tag) and nonce, it is mathematically insignificant when dealing with larger medical files.

C. Key Round and Size Effect

The key size analysis (Fig. 6) shows that the latency slightly increases as security is increased to 256-bit keys, which is approximately 34ms to 36ms with a 10 MB payload. This is because AES-256 (14 rounds) has more transformation rounds than AES-128 (10 rounds). Considering the sensitivity of medical PII, the slight performance sacrifice of the 256-bit security margin is worthwhile guaranteeing the protection of the data in the long term.

D. Summary of Trade-offs

<i>Feature</i>	<i>ECB</i>	<i>CBC</i>	<i>CTR</i>	<i>GCM</i>
Security	Low (Pattern Leakage)	High	High	High
Parallelization	Yes	No (Decryption only)	Yes	Yes
Integrity	No	No	No	Yes (AEAD)
Throughput	High	Low	Very High	High

VIII. CONCLUSION AND FUTURE WORK

In this experimental research, the four AES modes of operation were systematically compared (ECB, CBC, CTR and GCM) in terms of their performance, security and computational overhead in the environment of secure telemedicine image transmission architecture. Empirical benchmarking showed that though ECB has the lowest computational cost, it is deterministic, and thus cryptographically insecure with structured data, which has been conclusively demonstrated by the visible pattern leakage of encrypted medical images. CBC mode has been shown to be highly effective in preventing structural leakage by cryptographic chaining but suffers a major sequential bottleneck when encrypting high-resolution images, which restricts its scalability. On the other hand, stream-based modes (CTR and GCM) are taking advantage of parallelizable counter mechanisms to attain high throughput. Of these, GCM offers a balance between performance and security, with built-in Authenticated Encryption with Associated Data (AEAD), an authentication tag of 16 bytes, with a negligible payload overhead.

The key size analysis also affirmed that increasing the AES-128 to AES-256 incurs only a relatively small increase in latency (like 35 ms more per 10 MB payload) and significantly enhances the cryptographic safety margin, doubling the number of transformations rounds to 14. Considering these empirical results, we propose using AES-256-GCM as the underlying cryptographic mechanism in the proposed telemedicine architecture. It also aligns with the fundamental security definitions, namely confidentiality, integrity, authentication and high availability, assuring that clinical operations continue without interruption, and that healthcare data protection standards are meticulously followed.

Limitations and Future Work: This experiment was run in a single-node, controlled setting with synthetic random payloads and a Python-implemented cryptographic prototype. Future investigations ought to expand this assessment to hardware-accelerated settings (e.g., AES-NI instruction sets, Hardware Security Modules), into real-world network latency emulations, and streaming DICOM/TLS pipeline implementations. Also, with the progress of quantum computing, exploring post-quantum cryptographic solutions to long-term storage of sensitive medical data is an urgent research direction in the future.

IX. REFERENCES

Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. (2007, November). National Institute of Standards and Technology. Retrieved April 11, 2026, from <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>

High performance AES-GCM implementation based on efficient AES and FR-KOA multiplier. (2024, March). IEICE Electronics Express. Retrieved April 5, 2026, from https://www.jstage.jst.go.jp/article/elex/15/14/15_15.20180559/pdf/-char/en

Analyzing the security and performance trade-offs of common symmetric encryption algorithms in communication systems. (2026, February). ResearchGate. Retrieved April 6, 2026, from https://www.researchgate.net/publication/400933432_Analyzing_the_Security_and_Performance_Trade_offs_of_Common_Symmetric_Encryption_Algorithms_in_Communication_Systems

A Comparative study of AES encryption Modes and hashing for Blockchain Applications - Configuration manual. (2025b, June 3). NORMA@NCI Library - National College of Ireland. Retrieved April 10, 2026, from <https://norma.ncirl.ie/7729/>

Barker, E., Chen, L., Cooper, D., Moody, Regenscheid, A., Souppaya, M., Newhouse, W., Housley, R., Turner, S., Barker, W., & Kent, K. (2025, December 19). Considerations for achieving Crypto agility: Strategies and practices. National Institute of Standards and Technology. Retrieved April 12, 2026, from <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.39.pdf>